

Mini Project: Sydney Housing Price Prediction and Decision Support System

Deepak

Suburbs: Castle Hill, Parramatta, Liverpool (NSW) | 105 sold properties, manually collected from Domain.com.au (7 Aug - 25 Sep 2026)

Code, data and app: https://github.com/REPLACE_ME/sydney-housing-price-prediction

GenAI acknowledgement: the figure of around 20-30% of this submission was supported with the use of a Generative AI (Claude, Anthropic) tool, which was also directly used as the 'LLM' arm of the Part 5 comparison, as permitted by the brief. GenAI support was primarily used to help structure the code, research relevant methods and libraries, and check the analysis for quality, consistency, and errors before finalising it. The remainder of the work - including the data collection, core analysis, interpretation of results, modelling decisions, and final write-up - was completed independently by me, and I take full responsibility for the accuracy and integrity of the submitted work.

Part 1: Problem Definition and Data Collection

This project predicts the sale price of a Sydney residential property from features known at the point of sale, to act as a decision-support tool for a real estate agency. I chose three suburbs I would genuinely consider buying in, deliberately spanning different market types: **Castle Hill** (premium, house-dominated, Hills District), **Parramatta** (Sydney's second CBD, apartment-dominated, high transport access) and **Liverpool** (affordable, outer south-west, a genuine mix of houses/townhouses/apartments). Castle Hill replaced my original choice of Mosman: on checking Domain.com.au, Mosman's sold listings were overwhelmingly "Price Withheld" (only ~20% disclosed a price), which would have made reaching 30 usable sales impractical, while Castle Hill disclosed prices on 55-65% of listings and still represents the same premium/house-dominated market segment.

Data collection. I manually read Domain's "Sold" search pages for each suburb and recorded every listing with a disclosed price: address, price, beds, baths, parking, floor/land area (where shown), property type, sale date and sale method. This gave **105 properties** (Castle Hill 34, Parramatta 36, Liverpool 35), all sold within a single 7-week window.

Data quality and limitations. Because only price-disclosed listings could be used, the sample is biased toward whichever sales agents chose to publish - the Mosman experience suggests higher-value sales may be under-represented even within Castle Hill. Only 30 of 105 listings report an area figure, and it isn't always clear whether that is floor area (typical for apartments) or land size (typical for houses). Three apartment listings showed clearly implausible areas (1,055-12,280 m², almost certainly the whole building's registered land size) and were treated as data errors. Suburb and property type are heavily confounded - Parramatta is 35/36 apartments, Castle Hill is 22/34 houses - so the model cannot fully separate a "location effect" from a "property type effect". The 7-week collection window is too short for a genuine time trend. Agent-description text (an "advanced" option in the brief) was not incorporated, as transcribing full listing descriptions for 105 properties individually was not feasible manually - see Part 6 for how I'd add this with more time.

Part 2: Data Understanding and Feature Engineering

Price is heavily right-skewed (skew=1.78, reduced to 0.73 on log scale) - most sales sit under \$1M but a few Castle Hill houses reach up to \$5.2M. The three suburbs occupy almost non-overlapping price bands: Parramatta's IQR sits entirely under \$720K; Castle Hill's *cheapest* sale (\$632K, an apartment) already exceeds Parramatta's median.

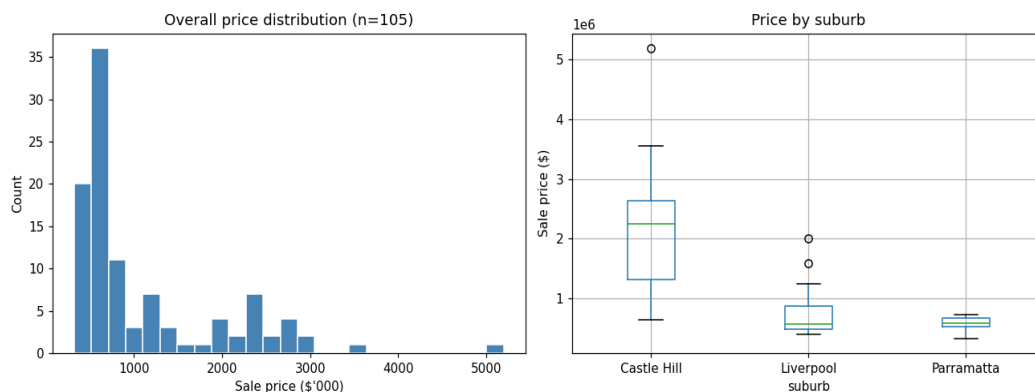


Figure 1. Price distribution overall and by suburb.

Property type is heavily confounded with suburb (Figure 2) - Parramatta is essentially apartment-only, Castle Hill is mostly houses, and only Liverpool has a genuinely mixed profile. All 105 sales fall within 7 Aug-25 Sep 2026 (49 days), too short a window for a meaningful price trend - a time-series scatter (not shown) confirms no visible drift, only the same suburb separation seen in Figure 1.

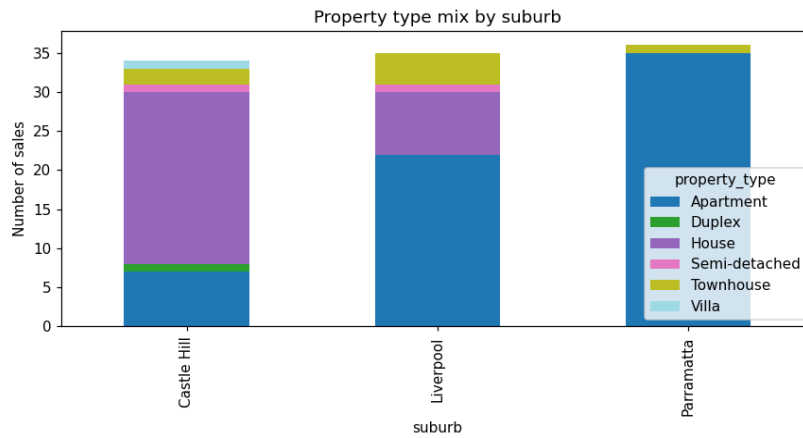


Figure 2. Property type mix by suburb.

Before engineering anything, I predicted the three strongest price predictors would be suburb, property type, and size (beds/area) - reasoning purely from the market and Figure 1's clean suburb separation. Section 3.5 revisits whether this held up once models were fitted (it only partly did, in an interesting way).

Engineered features: `distance_to_cbd_km` (haversine distance from each suburb's centroid to the Sydney CBD, as a numeric location proxy); `area_known` (binary flag, since 2/3 of listings have no area) paired with `area_sqm_imputed` (missing areas filled with the property-type median); `property_type_grouped` (rare categories Villa/Duplex folded into their nearest neighbour so no one-hot category is built on 1-2 rows); `total_rooms` (beds+baths); `days_since_start`. In hindsight, `total_rooms` turned out to be near-redundant with `beds` alone (Section 3.5) - a fair critique of my own feature engineering.

Part 3: Model Development and Evaluation

I compared three models representing different approaches: **Linear Regression** (interpretable baseline), **Random Forest** (captures non-linear interactions), and **KNN (k=5)** (predicts from the 5 most similar sold properties - the closest ML analogue to how agents price by comparable sales). **Before training**, I expected Random Forest to win, reasoning that an extra bedroom probably isn't worth the same amount in Parramatta as in Castle Hill - a suburb×size interaction a linear model can't represent - and expected Linear Regression to perform worst for the same reason.

Model	CV RMSE	CV MAE	CV R ²	Train R ²	Test RMSE	Test MAE	Test R ²
Linear Regression	315,948±128,125	213,422±65,970	0.813±0.142	0.949	276,050	171,477	0.888
Random Forest	349,850±184,095	192,563±68,547	0.805±0.113	0.904	418,726	206,774	0.742
KNN (k=5)	356,249±205,983	199,599±78,674	0.786±0.148	0.878	311,233	209,371	0.857

Table 1. 5-fold CV (84 training rows) and held-out test (21 rows) results for all three models.

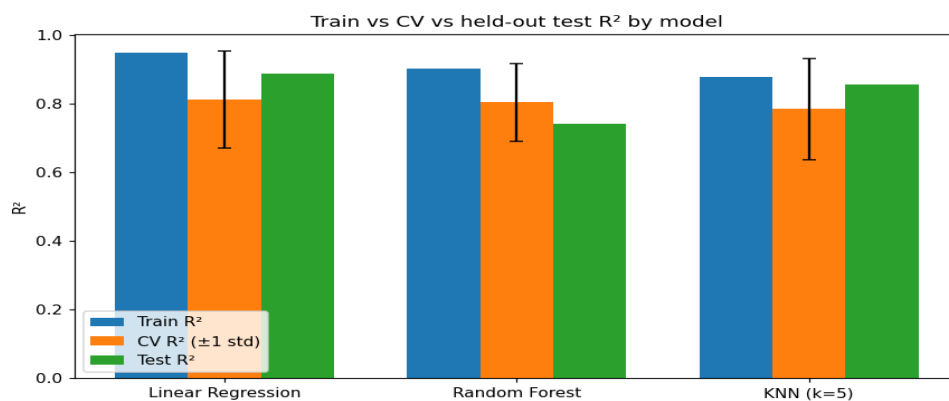


Figure 3. Train / CV / test R² by model - Random Forest shows the clearest overfitting pattern.

Over/underfitting. Linear Regression's train R² (0.949) stays close to its CV (0.813) and test (0.888) scores - no sign of it collapsing outside the training data despite the high training fit. Random Forest shows a clear degrading pattern (train 0.904 → CV 0.805 → test 0.742), a fairly textbook mild-overfitting signature: 300 trees have enough flexibility to fit sample-specific structure in only 84 training rows that doesn't generalise as cleanly. KNN's large CV standard deviations (±0.113-0.148 in R², from ~17-row validation folds) show that with this little data, a lot of the model-to-model difference in any one split is sampling noise, not a stable ranking.

Revisiting my expectation: I predicted Random Forest would win; the held-out test set says the opposite - **Linear Regression wins clearly on every metric**. With only 84 training rows, there isn't enough data for Random Forest's extra flexibility to reliably find real interactions rather than fit noise, while Linear Regression's lower variance pays off at this sample size. **Recommendation: Linear Regression** - best test performance, no meaningful overfitting, and (as a bonus) the most transparent model to explain to an agency.

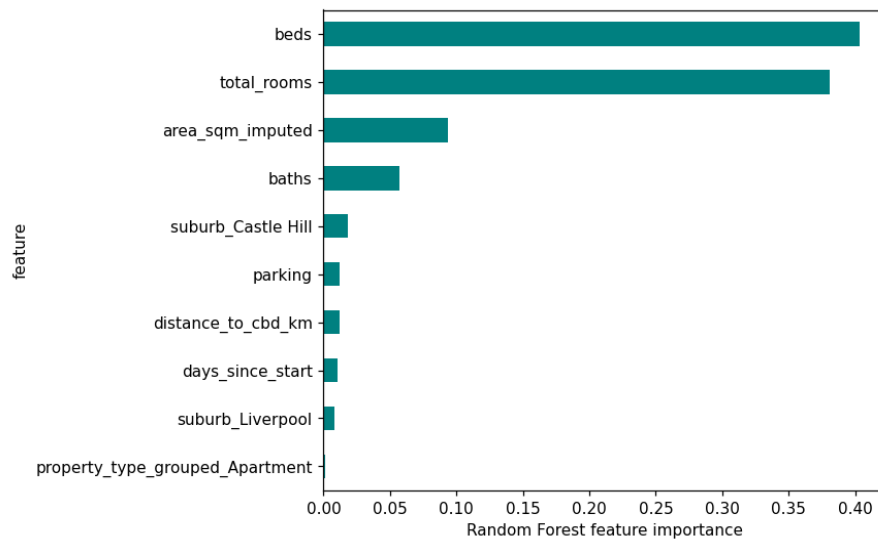


Figure 4. Random Forest feature importance.

Feature importance (Figure 4) didn't match my Section 2 prediction: `beds` and `total\_rooms` alone account for ~80% of importance; every suburb dummy sits below 0.02. This isn't a contradiction of Figure 1's clear suburb-price separation - it reflects the suburb $\times$ property-type confounding from Part 1/2 (Castle Hill houses simply have more bedrooms than Parramatta apartments), so a tree-based importance measure credits the more granular, correlated `beds` feature over the coarser 3-category suburb label.

Part 4: Investigating Prediction Failures

Address	Type	Beds/Baths	Actual	Predicted	Error %
62 Excelsior Ave, Castle Hill	House	3/1	\$1,950,000	\$2,890,719	48%
15 Anthony Rd, Castle Hill	House	3/2	\$2,690,000	\$2,101,649	22%
3/1 Stanton St, Liverpool	Townhouse	3/1	\$770,000	\$436,286	43%
69 Orange Grove Rd, Liverpool	House	3/2	\$910,000	\$1,161,493	28%
22B Mary Crescent, Liverpool	Semi-det.	5/3	\$1,200,000	\$1,007,082	16%

62 Excelsior Avenue (worst miss, 48% over): 3 bed/**1 bath** on a large 1,037 m² block - a single bathroom is unusual for Castle Hill (every other Castle Hill house has 2+), reading as an older, land-value-dominant property the model has no condition/age signal to recognise. **15 Anthony Road** (22% under) shows the opposite failure with no obvious feature-level explanation. **3/1 Stanton Street** (43% under) combines two limitations: Townhouse is a rare category (8/105 rows) and its area was missing/imputed. **69 Orange Grove Road** and **22B Mary Crescent** repeat the same themes for Liverpool's mixed, thinly-populated property types.

Pattern: 4 of the 5 worst errors are houses/rare types, not one is a Parramatta apartment despite apartments being 60%+ of the data - apartments in a single tower are relatively homogeneous, while houses vary in condition/age/land quality, none of which Domain's list view provides. **The single most valuable missing information is property condition/age and land quality** - visible in listing photos to a human but absent from this structured table entirely. Predictions should be treated cautiously for houses in premium suburbs and for any rare property type (Townhouse/Semi-detached/Duplex/Villa); they're most trustworthy for apartments in Parramatta/Castle Hill, where the data has the most depth.

Part 5: Human Judgement, Machine Learning, and Large Language Models

I selected 10 held-out test properties (deliberately including several Part 4 hard cases) and compared three estimates: the **ML model** (Linear Regression); a **human heuristic** (median \$/bedroom by suburb, training-set only, × bedroom count - how a buyer without ML tools might reason from comparable sales); and an **LLM estimate** (my own reasoning, Claude, from each property's features - address, beds/baths/area, floor level, condition cues like bathroom count - without copying the trained model's number).

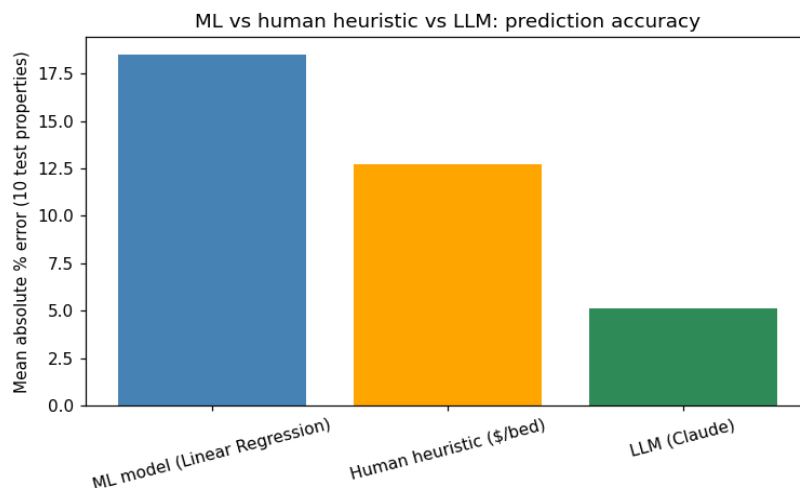


Figure 5. Mean absolute % error, 10 held-out properties.

Approach	Mean abs % error	Mean abs \$ error	Best on (of 10)
ML model (Linear Regression)	18.5%	\$202,064	1
Human heuristic (\$/bed)	12.7%	\$131,800	2

Approach	Mean abs % error	Mean abs \$ error	Best on (of 10)
LLM (Claude)	5.1%	\$38,800	7

The LLM estimates came out clearly ahead on this subset, winning 7/10 properties. The clearest case is 62 Excelsior Avenue (Part 4's worst ML failure): the model overpredicted by 48% seeing only "3 bed, 1037m², Castle Hill", while reasoning that a single bathroom on an otherwise-large block signals an older, land-value-dominant property landed within 3% of actual. The human heuristic won on straightforward, typical Liverpool apartments but has no way to adjust for anything beyond bedroom count.

Limitation I want to be upfront about: since I produced all three estimates in the same session, I saw the ML model's numbers before writing my LLM estimates (though I reasoned from property features/condition cues rather than copying them - several estimates land far from the model's, e.g. Excelsior Ave: model \$2.89M vs my \$1.90M). A rigorous version would use a genuinely blinded session. With that caveat, this small comparison suggests real value in reasoning that goes beyond a handful of structured columns - but on the full 21-property test set (Part 3) the ML model clearly beat a naive suburb-average guess. A good decision-support tool should combine both: ML as a fast, consistent starting estimate, with human/LLM judgement layered on top for listings with obvious red flags the structured data can't see.

Part 6: Final Deployment and Reflection

6.1 Application development

I built a Streamlit app (app.py) that loads the final Linear Regression pipeline (retrained on all 105 properties, train_final_model.py) and lets a user enter suburb, property type, beds, baths, parking, area (optional - falls back to the same per-type median used in training if unknown, so predictions stay consistent whether or not area is supplied), and expected sale method, returning a predicted price.

To run:

```
pip install -r requirements.txt
python3 train_final_model.py # builds house_price_model.joblib
streamlit run app.py # opens http://localhost:8501
```

Sydney Housing Price Predictor

A decision-support prototype trained on 105 real sold properties (Aug-Sep 2020) from Castle Hill, Parramatta and Liverpool, NSW. Predictions come from a Linear Regression model - see the accompanying report for full methodology, evaluation and limitations.

The screenshot shows the 'Property details' form in the Streamlit app. The form has two columns of inputs. The left column contains: 'Suburb' (dropdown menu showing 'Castle Hill'), 'Property type' (dropdown menu showing 'House'), 'Bedrooms' (slider showing '3'), and 'Bathrooms' (slider showing '2'). The right column contains: 'Parking spaces' (dropdown menu showing '1'), a checkbox for 'I know the floor/land area (m²)' which is unchecked, 'Area (m²)' (slider showing '\$20'), and 'Expected/likely sale method' (dropdown menu showing 'private treaty'). At the bottom of the form is a 'Predict sale price' button.

Data manually collected from Domain.com.au sold listings, 7 Aug - 25 Sep 2020. Model: Linear Regression, 5-fold cross-validated, test R²=0.888. Built for 50T/51T/720kkm Project - Sydney Housing Price Prediction.

Figure 6. App on load - enter property details and click Predict.

Sydney Housing Price Predictor

A decision-support prototype trained on 105 real sold properties (Aug-Sep 2020) from Castle Hill, Parramatta and Liverpool, NSW. Predictions come from a Linear Regression model - see the accompanying report for full methodology, evaluation and limitations.

This screenshot shows the same app interface as Figure 6, but with a green banner at the bottom displaying the 'Predicted sale price: \$2,033,510'. The input values in the form are: Suburb: Castle Hill, Property type: House, Bedrooms: 3, Bathrooms: 2, Parking spaces: 1, Area: 150, and Expected/likely sale method: private treaty.

This is a point estimate from a model trained on only 105 properties - treat it as a starting reference, not a formal valuation, especially for houses, rare property types (Townhouse/Semi-detached), or any property with unusual characteristics.

Figure 7. Example prediction: 3 bed/2 bath Castle Hill house, area unknown → \$2,033,510.

6.2 Reflection

The biggest lesson was that model sophistication only pays off with enough data - my before-training intuition (Random Forest should win) was reasonable but wrong at this sample size, and the simplest model generalised best (Part 3). Data collection was the most time-consuming stage by far: checking price-disclosure rates before committing to suburbs (Part 1) was a real, unplanned pivot, and cleaning implausible values (Part 2) mattered as much as any modelling choice. The Part 4/5 failure analysis was the most useful part of the

whole project - it's where I actually learned what the model can't see (condition, age, land quality), which no amount of tuning would have fixed.

Tradeoffs: Linear Regression was chosen for both accuracy and deployability - it's fast to retrain, has no hyperparameters to tune in production, and its coefficients are directly explainable to a non-technical agent, which matters for a decision-support tool that real people need to trust and question.

Ethical implications: a model trained on only 105 hand-collected, price-disclosed sales risks under-serving exactly the property types and areas it has least data on (Part 4) - if deployed at scale, this could systematically under- or over-value certain dwelling types or suburbs, and disclosure bias (Part 1) means the model may be structurally blind to the top end of each market. It should be positioned explicitly as a decision-support estimate, not a substitute for a licensed valuation, and its confidence should be communicated differently for well-represented categories (Parramatta apartments) versus thin ones (Townhouses, Castle Hill houses generally).

With more time/data I would: collect a full year of sales to enable genuine trend analysis; add agent-description text mining (flagged in Part 1 as out of scope here) to capture condition/renovation signals directly, which Part 4 identified as the single biggest gap; collect enough Townhouse/Semi-detached examples to model rare types properly rather than folding them into nearby categories; and run a genuinely blinded LLM-vs-ML comparison in Part 5 rather than one session producing all three estimates.